

Lab 1 — Packet Capture and Protocol Analysis with Wireshark

Cybersecurity Architecture (ACS) · Chapter 1

Lab 1: Packet Capture and Protocol Analysis with Wireshark

Objective

Learning Outcomes

Prerequisites

Tools Used

Part 0 — Setup (10 minutes)

Part 1 — First Capture and the Wireshark UI (15 minutes)

Part 2 — Intercepting a Ping (ICMP) (15 minutes)

Part 3 — Intercepting a DNS Request (20 minutes)

Part 4 — Intercepting HTTP Communication (25 minutes)

Part 5 — Export Your Evidence (5 minutes)

Fallback: `chapter1_reference.pcap`

Report Template

Challenge Questions

Deliverables

Grading Rubric

Troubleshooting

Cleanup

Further Reading

Lab 1: Packet Capture and Protocol Analysis with Wireshark

Course: Cybersecurity Architecture (ACS) — **Chapter 1** **Duration:** 90 minutes
Difficulty: Beginner — no prior Wireshark experience assumed

Objective

Capture live network traffic and dissect it layer by layer. You will intercept and analyse three different exchanges — an **ICMP ping**, a **DNS lookup**, and an **HTTP conversation** — and use them to show, with evidence from your own capture, why unencrypted protocols break the **confidentiality** and **integrity** properties of the CIA triad.

This is the practical counterpart to the Chapter 1 theory on the OSI/TCP-IP model and the CIA triad. Every packet you open is a layered object; your job is to learn to read it.

Learning Outcomes

After completing this lab you will be able to:

1. Select the correct capture interface and start/stop a capture in Wireshark.
 2. Map the panes of the Wireshark UI onto the layers of the OSI model.
 3. Distinguish a **capture filter** (pre-capture, BPF syntax) from a **display filter** (post-capture, Wireshark syntax) and explain why the difference matters.
 4. Identify and analyse ICMP echo request/reply pairs.
 5. Follow a DNS query to its answer and extract the resolved IP address.
 6. Reconstruct a full HTTP request/response using *Follow HTTP Stream*.
 7. Recover cleartext credentials from an HTTP POST and explain the security impact.
 8. Export a filtered capture and write up findings as evidence.
-

Prerequisites

- A Linux VM (**Kali Linux** or **Ubuntu 22.04+**) with a working network connection. Windows or macOS also work — only the installation commands differ.
- `sudo` / administrator privileges.
- Wireshark 4.x installed (see Part 0).
- A terminal and a web browser inside the same machine you capture on.

⚠ AUTHORISATION — read this before you start. Capture **only on your own machine, on your own traffic**. Do not capture on the school network, on a shared/public Wi-Fi, in promiscuous mode on a network you do not own, or on any interface carrying somebody else's traffic. Intercepting another person's communications without consent is a criminal offence in Belgium (Art. 314bis Sw.) and under the EU ePrivacy Directive. Everything in this lab is traffic **you generate yourself**.

Tools Used

Tool	Purpose
wireshark	GUI packet capture and analysis
tshark	Command-line Wireshark (used for verification)
ping	Generates the ICMP traffic
dig / nslookup	Generates the DNS traffic
curl or Firefox	Generates the HTTP traffic

Part 0 — Setup (10 minutes)

Step 0.1 — Install Wireshark

Kali / Debian / Ubuntu:

```
sudo apt update
sudo apt install -y wireshark tshark dnsutils curl
```

When the installer asks *"Should non-superusers be able to capture packets?"*, answer **Yes**.

If you missed that prompt, or you get "no interfaces found":

```
sudo dpkg-reconfigure wireshark-common      # answer Yes
sudo usermod -aG wireshark $USER
newgrp wireshark                           # or log out and back in
```

Step 0.2 — Verify the installation

```
wireshark --version | head -1
tshark -D          # lists capture interfaces
```

Expected output of `tshark -D` (names will differ on your machine):

1. eth0
2. any
3. lo (Loopback)

Step 0.3 — Identify your interface and addresses

```
ip -brief address
ip route | grep default
```

Write these down — you need them throughout the lab:

Item	Your value
Capture interface (e.g. eth0)	
Your IP address	
Your MAC address	
Default gateway IP	
Configured DNS server (<code>resolvectl status</code> or <code>cat /etc/resolv.conf</code>)	

Part 1 — First Capture and the Wireshark UI (15 minutes)

Step 1.1 — Start a capture

1. Launch Wireshark: `wireshark &`
2. In the welcome screen, **double-click your interface** (the one with a moving sparkline). Packets start scrolling immediately.
3. Let it run for ~10 seconds, then press the **red square** (Stop).

Step 1.2 — Map the three panes to the OSI model

Click any packet. You now see three panes:

Pane	Name	What it shows
Top	Packet List	One line per frame: time, source, destination, protocol, summary
Middle	Packet Details	The protocol tree — <i>this is the OSI stack, expanded</i>
Bottom	Packet Bytes	The raw hex and ASCII actually on the wire

Expand every line in the middle pane for one TCP packet. You will see roughly:

```

> Frame 12: 74 bytes on wire          <- capture metadata
> Ethernet II, Src: .., Dst: ..      <- Layer 2  (Data Link)
> Internet Protocol Version 4, Src: .., Dst: ..  <- Layer 3  (Network)
> Transmission Control Protocol, Src Port: ..    <- Layer 4  (Transport)
> Hypertext Transfer Protocol        <- Layer 7  (Application)

```

 **Task 1.1** — Screenshot this expanded tree. In your report, label each line with its OSI layer number and name. This screenshot is a graded deliverable.

Step 1.3 — Capture filters vs display filters

This distinction is examinable. Learn it now.

	Capture filter	Display filter
Applied	Before capture, in the kernel	After capture, in the UI
Syntax	BPF / tcpdump	Wireshark
Example	host 1.1.1.1 and icmp	ip.addr == 1.1.1.1 && icmp
Where	Welcome screen, or Capture > Options	Green bar above the packet list
Discarded traffic	Gone forever	Still in the file, just hidden
Use when	You know exactly what you want and the link is busy	Always — it is non-destructive

Rule of thumb: capture broadly, filter on display. You cannot go back and un-filter a capture filter.

 **Task 1.2** — Write, in your report, one scenario where a *capture* filter is genuinely the right choice, and explain why a display filter would not do.

Part 2 — Intercepting a Ping (ICMP) (15 minutes)

ICMP is a **Layer 3** protocol. It rides directly on IP — there is no TCP or UDP underneath it, and therefore no port numbers. This is the simplest exchange you will capture, which makes it the right place to start.

Step 2.1 — Start capturing

1. Start a new capture on your interface (**Capture > Start**, or the blue fin).

2. Leave the capture filter **empty** — you will filter on display.

Step 2.2 — Generate the traffic

In a terminal, while the capture is running:

```
ping -c 4 8.8.8.8
```

Expected output:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=12.4 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=11.9 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=12.1 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=12.0 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
```

Stop the capture.

Step 2.3 — Apply the display filter

Type into the green filter bar and press Enter:

```
icmp
```

You should see **8 packets**: four **Echo (ping) request** and four **Echo (ping) reply**.

Narrow it further — only the requests you sent:

```
icmp.type == 8
```

Only the replies:

```
icmp.type == 0
```

Only this conversation:

```
icmp && ip.addr == 8.8.8.8
```

Step 2.4 — Dissect one echo request

Select the **first** **Echo (ping) request** and expand the tree.

In the IPv4 layer, find and record:

- Source address, Destination address
- **TTL** (Time To Live)
- **Protocol:** ICMP (1)
- Header checksum

In the ICMP layer, find and record:

- **Type:** 8 (Echo (ping) request) and **Code:** 0
- **Identifier** and **Sequence number**
- The **Data** field (56 bytes)

Now select the matching **reply** and compare.

 **Task 2.1** — Complete this table from your own capture:

Field	Echo request	Echo reply
Source IP		
Destination IP		
ICMP Type	8	0
ICMP Code		
Identifier		
Sequence number		
TTL		
Frame length (bytes)		

Step 2.5 — Two things to notice

1. **The TTL differs between request and reply.** Your outgoing TTL is a round number (64, 128 or 255 depending on OS). The reply's TTL is that round number *minus the number of routers it crossed on the way back*.

 **Task 2.2** — From the reply's TTL, how many hops away is the server? Verify with `tracert 8.8.8.8` (or `tracert` on Windows) and state whether the two numbers agree. If they do not, give one reason why.

2. **The payload is not random.** Expand the ICMP `Data` field and read the ASCII in the bottom pane. Linux `ping` fills it with a timestamp followed by the byte pattern `0x10 0x11 0x12 ...`.

 **Task 2.3** — The request payload and the reply payload are **identical**. Why does the responder echo the data back instead of sending an empty reply? What could an attacker do with a protocol that copies arbitrary attacker-supplied data back? (Search term: *ICMP tunnelling*.)

Part 3 — Intercepting a DNS Request (20 minutes)

DNS normally runs over **UDP port 53**. It is the lookup that happens *before* almost every connection you make — and, classically, it is unauthenticated and unencrypted.

Step 3.1 — Flush the DNS cache first

If the answer is already cached, nothing goes on the wire and you will capture nothing. Clear it:

```
# systemd-resolved (most modern Ubuntu/Kali):
sudo resolvectl flush-caches

# if you use nscd instead:
sudo systemctl restart nscd 2>/dev/null

# verify the cache is empty
resolvectl statistics | grep -i "cache"
```

Step 3.2 — Capture the lookup

1. Start a new capture.
2. In a terminal, run:

```
dig neverssl.com
```

Expected output (abbreviated):

```
;; QUESTION SECTION:
;neverssl.com.          IN      A

;; ANSWER SECTION:
neverssl.com.          60      IN      A      34.223.124.45

;; Query time: 18 msec
;; SERVER: 192.168.56.1#53(192.168.56.1) (UDP)
```

1. Stop the capture.

Step 3.3 — Filter and analyse

```
dns
```

You should see a matched pair: **Standard query** and **Standard query response**.

Useful refinements:

```

dns.flags.response == 0      # queries only
dns.flags.response == 1      # responses only
dns.qry.name == "neverssl.com" # this lookup only
dns.qry.type == 1            # A records only (type 1)
udp.port == 53               # everything on the DNS port

```

Step 3.4 — Dissect the query

Select the **query** packet. Expand **Domain Name System (query)**.

Record:

- **Transaction ID** (the 16-bit value that matches query to response)
- **Flags** — confirm **Recursion desired: Set**
- **Questions: 1**, and under *Queries*: the **Name**, **Type (A)**, **Class (IN)**
- At the UDP layer: **source port** (random, high) and **destination port (53)**

Step 3.5 — Dissect the response

Select the **response** packet.

Record:

- **Transaction ID** — confirm it is **identical** to the query's
- **Flags** — confirm **Response: Message is a response** and **Reply code: No error (0)**
- **Answer RRs: count**
- Under *Answers*: the **Name**, **Type**, **TTL** and the resolved **Address**

 **Task 3.1** — Complete this table:

Field	Value from your capture
Queried domain name	
Query type	
Transaction ID	
Client source port	
DNS server IP	
Resolved IP address(es)	
Answer TTL (seconds)	
Response time (<code>dns.time</code>)	

Tip: add `dns.time` as a column — right-click the *Time* field inside the response packet's DNS tree and choose **Apply as Column**.

Step 3.6 — The security point

DNS over UDP/53 has **no authentication and no encryption**.

 **Task 3.2** — Answer in your report:

1. Your DNS query is readable by everyone on the path. Which property of the CIA triad does that violate?
 2. The **Transaction ID** is the only thing binding a response to a query. If an attacker can guess or observe it and reply faster than the real server, what attack have they performed, and what is it called?
 3. Name **two** protocols designed to fix this, and state what each one actually protects. (Hint: one encrypts the channel, one signs the data. They are not the same fix.)
-

Part 4 — Intercepting HTTP Communication (25 minutes)

HTTP over TCP port 80 is **cleartext**. This part is where Chapter 1's theory about confidentiality stops being abstract.

Step 4.1 — Capture a page load

1. Start a new capture.
2. Fetch a plain-HTTP page. `neverssl.com` exists precisely for this purpose — it never redirects to HTTPS:

```
curl -v http://neverssl.com/
```

Or browse to `http://neverssl.com` in Firefox.

1. Stop the capture.

Step 4.2 — The TCP three-way handshake

Before any HTTP appears, TCP must connect. Filter:

```
tcp.flags.syn == 1 || tcp.flags.fin == 1
```

Find the three packets that open the connection:

#	Direction	Flags	Meaning
1	client → server	[SYN]	"I want to talk, my sequence number is X"
2	server → client	[SYN, ACK]	"Accepted, mine is Y, I acknowledge X+1"
3	client → server	[ACK]	"I acknowledge Y+1" — connection established

 **Task 4.1** — Screenshot the three handshake packets and record the raw sequence numbers. (Wireshark shows *relative* sequence numbers by default. To see the real ones: **Edit > Preferences > Protocols > TCP**, untick *Relative sequence numbers*.)

Step 4.3 — The HTTP request and response

Filter:

```
http
```

You should see at least a `GET / HTTP/1.1` and an `HTTP/1.1 200 OK`.

More filters worth knowing:

```
http.request          # requests only
http.response         # responses only
http.request.method == "GET"
http.response.code == 200
http.host == "neverssl.com"
tcp.port == 80        # all traffic on the HTTP port
```

Select the `GET` packet and expand `Hypertext Transfer Protocol`. Record the **request line**, **Host**, **User-Agent**, and **Accept** headers.

Select the `200 OK` and record the **status line**, **Server**, **Content-Type** and **Content-Length** headers.

Step 4.4 — Follow the stream

Right-click the `GET` packet → **Follow > HTTP Stream**.

A window opens showing the **entire conversation reassembled as text**: your request in one colour, the server's response and the full HTML body in another.

Note the filter Wireshark wrote for you: `tcp.stream eq 0`.

 **Task 4.2** — Screenshot the Follow HTTP Stream window. In one sentence, explain what reassembly Wireshark performed to produce it — the HTML arrived split across multiple TCP segments, so what did Wireshark have to do?

Step 4.5 — Capturing credentials in cleartext

Now the point of the whole exercise.

1. Start a new capture.

2. Submit a login form over plain HTTP. Use the **deliberately vulnerable test endpoint** below — it is a public sandbox that accepts any credentials:

```
curl -X POST http://httpbin.org/post \
  -d "username=student" \
  -d "password=Vives2026!"
```

⚠ **Never use a real password here.** Use exactly the dummy value above. You are about to prove that anyone on the path can read it.

1. Stop the capture and filter:

```
http.request.method == "POST"
```

1. Select the POST packet, expand HTML Form URL Encoded: application/x-www-form-urlencoded.

The username and password are sitting there in plain text.

1. Right-click → **Follow > HTTP Stream** and read the whole exchange.

 **Task 4.3** — Screenshot the form fields visible in the packet details pane.

Then answer:

1. Which CIA property has been broken, and at which OSI layer was the data exposed?
2. Repeat the request against `https://httpbin.org/post` (note the **s**) and capture it. Filter with `tls`. Can you still read the credentials? What *can* you still see — and why is Server Name Indication (SNI) still visible in the Client Hello?
3. HTTPS fixes confidentiality here. Name one thing about this exchange that HTTPS does **not** hide from a network observer.

Step 4.6 — A faster way to find credentials

Wireshark ships a report that scans a whole capture for cleartext secrets:

Tools > Credentials

Open it on your capture. Anything it lists was transmitted in the clear.

The command-line equivalent, useful for large files:

```
tshark -r your_capture.pcapng -Y 'http.request.method=="POST"' \
-T fields -e urlencoded-form.key -e urlencoded-form.value
```

Part 5 — Export Your Evidence (5 minutes)

Your capture currently contains everything your machine did, including traffic unrelated to the lab. Export only what you need.

Step 5.1 — Export a filtered subset

1. Apply the display filter you want to keep, for example:

```
icmp || dns || http
```

1. **File > Export Specified Packets...**
2. Select **All packets** → **Displayed** (not *Captured*).
3. Save as `lastname_firstname_chapter1.pcapng`.

Step 5.2 — Verify the export

```
capinfos lastname_firstname_chapter1.pcapng
tshark -r lastname_firstname_chapter1.pcapng | head -40
```

Confirm your file contains **all three** required protocols:

```
tshark -r lastname_firstname_chapter1.pcapng -Y 'icmp' | wc -l    # must be > 0
tshark -r lastname_firstname_chapter1.pcapng -Y 'dns' | wc -l    # must be > 0
tshark -r lastname_firstname_chapter1.pcapng -Y 'http' | wc -l    # must be > 0
```

⚠ Privacy check before you submit. Your capture may contain traffic from other applications — mail clients, messengers, background sync. Filter it out before exporting, and never submit a capture containing a real credential of yours.

Fallback: `chapter1_reference.pcap`

If you cannot capture live traffic (no lab network, no privileges, broken VM), use the file `chapter1_reference.pcap` included alongside this lab. It contains the complete scenario — ARP, DNS, four pings, an HTTP GET with a 200 OK, and an HTTP POST carrying cleartext credentials — and every display filter in this document works on it unchanged.

```
wireshark chapter1_reference.pcap
```

This file is a **synthetic capture**, generated by `make_reference_capture.py` in this folder. It was assembled packet by packet, not sniffed from a real network, so no real person's traffic is in it. All checksums, sequence numbers and timings are valid, so Wireshark dissects and reassembles it exactly like a live capture.

If you use the reference file, say so in your report. Using it caps the *Capture setup* portion of your grade (see rubric) but costs you nothing elsewhere.

Report Template

Submit a single PDF using this structure.

Chapter 1 Lab Report – Packet Capture and Protocol Analysis

Name:

Student number:

Date of capture:

Capture interface / OS:

Used live capture or chapter1_reference.pcap:

1. Setup

- Interface, IP, MAC, gateway, DNS server
- Screenshot: expanded protocol tree with OSI layers labelled (Task 1.1)
- Capture vs display filter scenario (Task 1.2)

2. ICMP Analysis

- Screenshot: filtered ICMP packets
- Request/reply comparison table (Task 2.1)
- TTL and hop-count analysis (Task 2.2)
- Payload echo discussion (Task 2.3)

3. DNS Analysis

- Screenshot: query and response pair
- DNS field table (Task 3.1)
- Security discussion: CIA, spoofing, DoH/DoT/DNSSEC (Task 3.2)

4. HTTP Analysis

- Screenshot: TCP three-way handshake + sequence numbers (Task 4.1)
- Request and response headers, in a table
- Screenshot: Follow HTTP Stream + reassembly explanation (Task 4.2)
- Screenshot: cleartext credentials + CIA/HTTPS discussion (Task 4.3)

5. Conclusion

In 150–250 words: which of the three protocols you captured leaked the most information about you and your machine, and what a defender should deploy to close each gap.

6. Answers to Challenge Questions

1–10 below.

Challenge Questions

Answer all ten in your report.

1. What is the difference between a capture filter and a display filter? Give the syntax of each for "only ICMP traffic to 8.8.8.8".
2. ICMP has no port numbers. Why not, and what does that tell you about which OSI layer it operates at?
3. In your ping capture, why does the TTL of the reply differ from the TTL of your request?
4. What is the purpose of the DNS Transaction ID, and how does an attacker abuse it?
5. Your DNS query used UDP. Under what circumstances does DNS switch to TCP?
6. Describe the TCP three-way handshake. Why is a two-way handshake insufficient?

7. What does `tcp.stream eq 0` mean, and how did Wireshark know which packets belong together?
 8. You captured a password in cleartext. Name the OSI layer at which the exposure occurred and the OSI layer at which HTTPS fixes it.
 9. Compare the same request over HTTP and HTTPS in Wireshark. List three things still visible to an observer after TLS is applied.
 10. You are asked to detect cleartext credential submissions across a campus network. Which tool from this lab scales to that job, and what is the single biggest legal constraint on doing it?
-

Deliverables

Submit **one ZIP file** named `lastname_firstname_chapter1.zip` containing:

1. **report.pdf** — the lab report, following the template above.
 2. **lastname_firstname_chapter1.pcapng** — your exported capture, containing ICMP, DNS and HTTP.
 3. **screenshots/** — the five required screenshots (Tasks 1.1, 4.1, 4.2, 4.3, plus your filtered ICMP view), as PNG.
 4. **lab_notes.txt** — the commands you actually ran, and any errors you hit and how you solved them.
-

Grading Rubric

Criterion	Weight	What earns full marks
Capture setup & interface selection	10%	Correct interface, all four setup values recorded, live capture performed
ICMP analysis	20%	Complete table, correct TTL/hop reasoning, payload question answered
DNS analysis	20%	Complete table, query/response correctly matched by Transaction ID
HTTP analysis	25%	Handshake identified, stream followed, cleartext credentials demonstrated
Filter usage	10%	Correct display filters shown in screenshots, capture vs display understood
Report quality & evidence	10%	All screenshots present, legible and labelled; conclusion is specific
Challenge questions	5%	All ten answered correctly

Deduction: using `chapter1_reference.pcap` instead of a live capture forfeits the *Capture setup* criterion (10%). Everything else is markable as normal.

Troubleshooting

Problem	Cause	Fix
No interfaces listed in Wireshark	User not in the <code>wireshark</code> group	<code>sudo dpkg-reconfigure wireshark-common</code> then <code>sudo usermod -aG wireshark \$USER</code> , log out and in
You don't have permission to capture	Same as above	As above, or run <code>sudo wireshark</code> (not recommended)
No DNS packets appear	Answer served from cache	<code>sudo resolvectl flush-caches</code> , then retry
DNS query appears but no response	Firewall dropping UDP/53, or DoH enabled in the browser	Use <code>dig</code> from the terminal, not the browser
<code>http</code> filter shows nothing	The site redirected to HTTPS	Use <code>http://neverssl.com</code> — it never redirects
Only TLS packets, no HTTP	Browser forced HTTPS-Only mode	Disable HTTPS-Only mode, or use <code>curl</code>
Capture is enormous	Capturing all interfaces (<code>any</code>)	Pick one specific interface
Can't read the HTTP body	Response was gzip-compressed	Follow HTTP Stream decompresses it; or <code>curl -H "Accept-Encoding: identity"</code>
Filter bar is red	Display filter syntax error	Red = invalid, green = valid. Check <code>==</code> vs <code>=</code>

Cleanup

```
# Remove capture files containing your own traffic once submitted
rm -f ~/lab_capture_*.pcapng

# If you added yourself to the wireshark group only for this lab:
# sudo gpasswd -d $USER wireshark
```

Do not leave capture files with real traffic on shared lab machines.

Further Reading

- Wireshark User's Guide: https://www.wireshark.org/docs/wsug_html_chunked/
- Display Filter Reference: <https://www.wireshark.org/docs/dfref/>
- Wireshark sample captures: <https://wiki.wireshark.org/SampleCaptures>
- RFC 792 — ICMP: <https://www.rfc-editor.org/rfc/rfc792>
- RFC 1035 — DNS: <https://www.rfc-editor.org/rfc/rfc1035>
- RFC 9110 — HTTP Semantics: <https://www.rfc-editor.org/rfc/rfc9110>

Course: Cybersecurity Architecture (ACS) — Chapter 1 **Author:** Milan Dima — milan.dima@vives.be **Licence:** CC BY 4.0 **Last updated:** 30 September 2026